

Aletheia Three.js Animation - How-to for Hobby Developers

This guide explains how the interactive Aletheia star animation was developed and how its main parts work together.

The final browser app lives at:

`aletheia-app/aletheia-threejs-animation/aletheia-threejs-animation.htm`

The companion recreation prompt is: `aletheia-threejs-animation-code.md`

The reusable knowledge file is: `aletheia-threejs-animation-knowledge.md`

The Storyteller version is: `stories/aletheia-threejs-animation-story.md`

The app combines Three.js, WebGL point rendering, Canvas 2D, CSS 3D transforms, and ordinary HTML controls.

1. Architecture overview

The most important architectural decision is that the app does not create a different particle system for Spiral, AI and Aletheia. It creates one foreground point cloud and three destination maps. The same points move from one destination map to another.

Three.js is responsible for the real-time universe. Canvas 2D is used twice: first to paint the radial-gradient sun texture, and second to rasterise text so visible pixels can become particle targets. HTML and CSS handle the title panel, the Star-Wars-style crawl, and all debug controls.

This division keeps each technology doing the job it is good at. WebGL moves thousands of points efficiently. Canvas provides easy pixel access. CSS keeps text sharp and editable.

2. The renderer, scene and camera

The page creates a `THREE.WebGLRenderer` with antialiasing and alpha enabled. The renderer draws into the full browser window.

The renderer pixel ratio is capped at 2. That prevents very high-DPI screens from multiplying the GPU workload without a useful visual gain.

A `THREE.Scene` stores the background stars and the animated universe group. A `THREE.PerspectiveCamera` supplies depth and perspective.

The animated foreground is placed inside a `THREE.Group` called `universe`. Rotating that group rotates the complete star system and the central sun together.

On window resize the renderer size and camera aspect ratio are recalculated.

3. Building the particle point cloud

The main foreground cloud uses about 5,600 points. Typed `Float32Array` buffers store position,

size and colour.

Position uses three floats per particle: x, y and z. Colour uses three floats per particle: red, green and blue. A custom `aSize` attribute stores point size.

These arrays are attached to `THREE.BufferGeometry`. `THREE.Points` draws one point for every geometry vertex.

A custom `THREE.ShaderMaterial` turns each square point primitive into a round luminous star. The fragment shader measures distance from the centre of `gl_PointCoord` and fades the edge.

`THREE.AdditiveBlending` is important. Where particles overlap, their light accumulates. Dense clusters therefore appear brighter and more star-like.

4. How the spiral is generated

The spiral is a visual construction rather than a physically accurate galaxy simulation.

The current design uses three principal arms. A proportion of points are shifted into a nearby sibling branch so a main arm can appear to split shortly after leaving the centre.

A distance parameter `t` runs from approximately 0 near the centre to 1 at the outer edge. Radius increases with `t`.

The key idea is that angular curvature decreases as distance increases. The code uses a term similar to: `curve = thetaMax * Math.pow(1 - t, 1.28)`.

Near the centre, `1 - t` is large, so the arm receives strong curvature. Near the outside, `1 - t` approaches zero, so the direction changes less and the arm becomes straighter.

Small Gaussian random offsets widen each arm and prevent it from looking mechanically perfect. A small `z` spread gives the spiral genuine 3D depth when tilted.

5. How the sun is made

The central sun is deliberately inexpensive. It is not a sphere mesh or a volumetric lighting simulation.

A hidden 256 by 256 Canvas 2D surface paints a radial gradient. The centre is white, then warm cream, then pale blue glow, then transparency.

That canvas becomes a `THREE.CanvasTexture`. The texture is placed on `THREE.SpriteMaterial` and then on a `THREE.Sprite`.

A sprite always faces the camera, so the sun remains a convincing glowing disc while the universe group rotates.

The preferred current default sun scale is 4.0. The debug panel can resize it or hide it completely.

A tiny opacity and size pulse is added during animation to stop the sun feeling completely static.

6. Turning AI into star text

AI is not built from font geometry. The word is first drawn in white onto an off-screen 2D canvas using a heavy sans-serif font.

`CanvasRenderingContext2D.getImageData()` returns the RGBA pixels. The code scans those pixels at a small step interval and records coordinates whose alpha channel is visible.

Those visible pixel positions form a pool of candidate destinations. Each of the 5,600 particles is assigned one of those destinations.

Canvas pixel coordinates are centred, scaled into the Three.js coordinate system, and given a tiny random z value.

Because a proper font is rasterised, the I has a top and base rather than appearing as an arbitrary rectangular bar.

7. Turning Aletheia into star text

Aletheia uses the same text-to-particle pipeline as AI.

The difference is typography and scale. The word remains Aletheia in Latin letters, but it uses Cinzel first, with classical serif fallbacks. This gives a Greek or classical visual character without changing the spelling into Greek characters.

Aletheia is much wider than AI, so its target coordinate width is larger. The preferred point-size setting of 0.30 helps the longer word remain legible.

The text target still has a small z thickness, so the word is not a perfectly flat sheet when the scene is tilted.

8. Morphing one cloud between shapes

The targets object holds three arrays: Spiral, AI and Aletheia. Every target contains the same number of x, y and z values.

Changing stage simply changes activeTarget. The visible geometry is not replaced.

During `requestAnimationFrame`, each current position moves a small fraction of the remaining distance to its target.

Conceptually: $\text{current} = \text{current} + (\text{target} - \text{current}) * \text{attraction}$.

The attraction value is based on frame delta time so motion is more consistent on fast and slow machines.

The transition is intentionally slow enough to let the viewer watch the stars travel and merge rather than seeing an instant swap.

Auto mode holds Spiral, AI and Aletheia for different intervals and then cycles back to

Spiral.

9. Drag rotation

Pointer dragging changes target x and y rotation values for the universe group.

The real rotation eases toward those target values instead of jumping directly. That gives the interaction a softer, inertial feel.

The three stages have slightly different preferred tilts. Spiral and AI can lean backward to show depth. Aletheia returns closer to a face-on view because the longer word is easier to read that way.

10. Pointer repulsion

Normal pointer movement pushes nearby particles away.

The difficult part is mapping a two-dimensional screen pointer into the three-dimensional particle coordinate system.

THREE.Raycaster creates a ray from the camera through the pointer position. The ray intersects an invisible plane passing through the universe centre and facing the camera.

The intersection point is converted into the universe group's local coordinate system.

Particles inside a small radius are nudged away from that point. This is a lightweight approximation rather than a full particle-physics simulation, but it creates the useful 'finger through star dust' effect.

11. Camera fitting and scale debugging

A major development lesson was that missing-looking geometry was often a scale problem rather than a rendering failure.

FIT UNIVERSE asks the BufferGeometry to compute its bounding sphere. The local radius is multiplied by the current universe scale.

The camera field of view is converted to radians. From the world radius and half field of view, the code estimates a camera z distance that fits the cloud with a small margin.

Current preferred defaults are Universe 1.00, Particles 0.30, Camera 0.50 and Sun 4.0.

The quick universe buttons are 0.5x, 0.7x and 1x. An earlier 10x experiment was removed because it encouraged confusing scales.

12. The Star-Wars-style crawl

The gold crawl is ordinary HTML and CSS layered above the WebGL canvas. It is not Three.js geometry.

The wrapper is fixed over the scene and remains transparent, so it never creates an opaque black rectangle over the stars.

CSS perspective is applied to the wrapper. The crawl text is rotated backward with `rotateX` and translated upward by a keyframe animation.

The combination makes the words appear to travel away toward the central sun.

The crawl height is clipped so the text does not continue indefinitely past the centre.

The first Aletheia heading is intentionally large and bold. Let AI do the walking is the next major line.

The crawl then references the Aletheia and Thalia protocols, practical tools, stories, interfaces, local knowledge, prototypes, and the three GitHub project paths.

13. Top-left title and Debug button

The public interface is deliberately small. A fixed HTML panel at the top left shows Aletheia Three.js Animation, one instruction line, and a Debug button.

Keeping this panel outside WebGL means it remains crisp at any canvas resolution and the button stays a normal accessible HTML control.

Debug is hidden by default. The button toggles a debug-on class on `document.body`.

CSS selectors use that class to reveal or hide the scale panel, stage badge, bottom stage controls and bottom-right summary.

14. Bottom-left debug panel

When Debug is enabled, the bottom-left panel exposes the variables that proved most useful during development.

Universe scales the complete Three.js universe group.

Particles changes the point-size shader uniform.

Camera changes the fitted viewing-distance multiplier.

Sun Size changes the sprite scale, and a checkbox can turn the sun off.

FIT UNIVERSE reframes the current geometry. RESET VIEW restores the preferred values.

A diagnostic block underneath reports stage, universe scale, particle scale, camera z, sun size, local radius, world radius and total point count.

15. Bottom-middle controls

The middle debug buttons can force Spiral, AI or Aletheia directly.

Auto turns the timed cycle on or off. Replay restarts from Spiral. Crawl shows or hides the

gold perspective text. Fullscreen uses the browser Fullscreen API.

These direct controls make development faster because a developer can freeze the animation on one target while tuning its appearance.

16. Bottom-right live debug summary

The bottom-right line is a compact summary such as: Debug - Universe 1.00 / Particles 0.30 / Camera 0.50 / Sun 4.0.

An earlier version hard-coded those words. Moving a slider changed the animation but not the summary.

The corrected version generates the string from live variables and calls `updateDebugHint` whenever a relevant control changes.

This is a general UI lesson: if a label represents state, derive the label from the state instead of maintaining a second manual copy.

17. Performance and portability

The app is modest by WebGL standards. Roughly 5,600 foreground particles are updated on the CPU and drawn as one Points object. Background stars live in a second, mostly static point cloud.

Performance choices include capping pixel ratio at 2, reusing one point cloud for all stages, using typed arrays, using a sprite rather than complex sun geometry, and keeping readable text in HTML/CSS.

A future offline edition could store Three.js and the Cinzel font locally. A future accessibility pass could add `prefers-reduced-motion` support and a no-crawl option as the default for users who request reduced motion.

18. Development lessons

Check scale before assuming geometry has vanished.

Let the desired visual result guide the mathematics. The useful spiral was the one whose arms looked right, not the one with the neatest formula.

Use each browser technology for its strengths: WebGL for many moving points, Canvas for pixels and textures, CSS for typography and perspective, HTML for controls.

Keep debug controls while developing, then hide them for normal users.

Make status displays live rather than hard-coded.

Small tricks combine well: sampled text pixels become a star font, a radial gradient becomes a sun, and CSS transforms become a cinematic crawl.

19. Reference URLs

Three.js documentation: <https://threejs.org/docs/>

WebGLRenderer: <https://threejs.org/docs/pages/WebGLRenderer.html>

Points: <https://threejs.org/docs/pages/Points.html>

BufferGeometry: <https://threejs.org/docs/pages/BufferGeometry.html>

ShaderMaterial: <https://threejs.org/docs/pages/ShaderMaterial.html>

PerspectiveCamera: <https://threejs.org/docs/pages/PerspectiveCamera.html>

Raycaster: <https://threejs.org/docs/pages/Raycaster.html>

SpriteMaterial: <https://threejs.org/docs/pages/SpriteMaterial.html>

Canvas fillText:

<https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/fillText>

Canvas getImageData:

<https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/getImageData>

requestAnimationFrame:

<https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame>

CSS perspective: <https://developer.mozilla.org/en-US/docs/Web/CSS/perspective>

Fullscreen API: https://developer.mozilla.org/en-US/docs/Web/API/Fullscreen_API